

Vue components provide functionality that pure custom elements do not, most notably cross-component data flow, custom event communication, and build tool integrations.

While Vue does not internally use custom elements, it has excellent interoperability when consuming or delivering custom elements. Vue CLI also allows you to create Vue components that file as native custom elements.

When a Vue instance is created, it must go through a sequence of initialization steps, such as setting up data observation, compiling the prototype, and creating the appropriate data bindings. Along the way, it will also call some lifecycle hooks, which will enable us to execute custom logic. Other hooks will be named at various stages of the instance's lifecycle, such as compiled, primed, and demolished. Both lifecycle hooks are named with this meaning, referring to the Vue instance that invoked it. Any users may be asking where the definition of “controllers” exists in the universe of Vue.js, and the response is: there are no controllers in Vue.js. Your component's custom logic will be distributed through these life-cycle hooks.

Another interesting principle in Vue is the component framework, an abstraction that helps one construct large-scale applications out of lightweight, self-contained, and mostly reusable modules. Almost any kind of application interface can be abstracted into a tree of components if we think about it.

CODE:

```
• var data = { a: 1 }
•
• // The object is added to a Vue instance
• var vm = new Vue({
•   data: data
• })
•
• // Getting the property on the instance
• // returns the one from the original data
• vm.a == data.a // => true
•
• // Setting the property on the instance
• // also affects the original data
• vm.a = 2
• data.a // => 2
•
```